CS4745 Assignment 3

Antoine Giroux 3616164

2026/04/07

Design Summary

Adaptive Integration Method

This allows high precision in high-frequency regions while skipping unecessary calculations in low-frequency regions. For any interval [a,b] this algorithm first calculates the Simpson's estimate and compares it to the sum of the estimate of the two halves, if the difference exceeds the tolerance then it recursively subdivides the regions.

MPI Dynamic Design

Master

The master maintains a queue of tasks, of size 64 which is enough for each worker to have work, it then listens for two types of messages from workers: - RESULT: The worker has finished and gives the Master a value to be added to the global sum - WORK: A worker has encountered a high error region and is adding more work to the queue.

Worker

The worker listens for tasks, performing one level of the adaptive split. If more recursion is needed it sends the right half back to the master as more work to be given to idle processes and processes the left half itself through recursion, aiming to minimize the ammount of time a processor is idle.

MPI Hybrid Design

Like in Mode 1 the interval 0 -> 1 is split into K intervals but instead of making use of dynamic allocation each worker calculates their assigned intervals locally making use of OpenMP to parallelize the recursion using tasks in each process.

OpenMP Design

Result tables

mode	func	tol	procs (OpenMP)	threads (MPI)	workers	time	speedup	efficiency
Mode 0	0	1e-6	1	1	1	0.000034111	1.0	1.0
Mode 0	0	1e-8	1	1	1	0.000032888	1.0	1.0
Mode 0	1	1e-6	1	1	1	0.000004777	1.0	1.0
Mode 0	1	1e-8	1	1	1	0.000006888	1.0	1.0
Mode 0	2	1e-6	1	1	1	0.000352000	1.0	1.0
Mode 0	2	1e-8	1	1	1	0.001118555	1.0	1.0
Mode 1	0	1e-6	1	1	1	0.000084444	0.4039474494	0.4039474494
Mode 1	0	1e-6	2	1	2	0.001299666	0.2624603870	0.0131230193
Mode 1	0	1e-6	4	1	4	0.001049555	0.3250052040	0.0081251301
Mode 1	0	1e-6	8	1	8	0.001109888	0.3073379960	0.0038417249
Mode 1	0	1e-6	16	1	16	0.001485333	0.2296528320	0.0014353302
Mode 1	0	1e-8	1	1	1	0.000074222	0.4431127075	0.4431127075
Mode 1	0	1e-8	2	1	2	0.001218000	0.2700229880	0.0135011494
Mode 1	0	1e-8	4	1	4	0.000975888	0.3370138070	0.0084253451
Mode 1	0	1e-8	8	1	8	0.001421666	0.2313397520	0.0028917469
Mode 1	0	1e-8	16	1	16	0.001498555	0.2194700160	0.0013716876
Mode 1	1	1e-6	1	1	1	0.000073777	0.0647580501	0.0647580501
Mode 1	1	1e-6	2	1	2	0.001308333	0.0036517453	0.0018258726
Mode 1	1	1e-6	4	1	4	0.000966444	0.0049435849	0.0012358962

mode	func	tol	procs (OpenMP)	threads (MPI)	workers	time	speedup	efficiency
Mode 1	1	1e-6	8	1	8	0.0013913333	3.0034339004	0.0004292375
Mode 1	1	1e-6	16	1	16	0.0013447777	7.0035527805	0.0002220487
Mode 1	1	1e-8	1	1	1	0.0000794444	0.8671221630	0.0867122163
Mode 1	1	1e-8	2	1	2	0.0017471111	1.0039429661	0.0019714830
Mode 1	1	1e-8	4	1	4	0.0013535555	5.0050894108	0.0012723527
Mode 1	1	1e-8	8	1	8	0.0016066666	4.0042876350	0.0005359543
Mode 1	1	1e-8	16	1	16	0.0019230000	7.0035823192	0.0002238949
Mode 1	2	1e-6	1	1	1	0.0000671111	5.2450339809	5.2450339809
Mode 1	2	1e-6	2	1	2	0.0054562222	2.0645135016	0.0322567508
Mode 1	2	1e-6	4	1	4	0.0042704444	4.0824270186	0.0206067546
Mode 1	2	1e-6	8	1	8	0.0055680000	8.0632183908	0.0079022988
Mode 1	2	1e-6	16	1	16	0.0063116666	16.055769739	0.0034856086
Mode 1	2	1e-8	1	1	1	0.0000673333	16.612218619	16.612218619
Mode 1	2	1e-8	2	1	2	0.0075536666	1.4808113180	0.0740405659
Mode 1	2	1e-8	4	1	4	0.0059145555	5.1891191146	0.0472797786
Mode 1	2	1e-8	8	1	8	0.0068980000	8.0162156494	0.0202695618
Mode 1	2	1e-8	16	1	16	0.0097486666	11.473933326	0.0071712082
Mode 2	0	1e-6	2	4	8	0.0018187777	7.0187549583	0.0023443697
Mode 2	0	1e-6	4	2	8	0.0013401111	1.0254539343	0.0031817417
Mode 2	0	1e-6	4	4	16	0.0012658888	8.0269463636	0.0016841477
Mode 2	0	1e-8	2	4	8	0.0018467777	7.0178087487	0.0022260935

mode	func	tol	procs (OpenMP)	threads (MPI)	workers	time	speedup	efficiency
Mode 2	0	1e-8	4	2	8	0.001434888	8.022920800	0.002865100
Mode 2	0	1e-8	4	4	16	0.001204222	9.552731123	0.001706952
Mode 2	1	1e-6	2	4	8	0.001608444	4.002970385	0.000371298
Mode 2	1	1e-6	4	2	8	0.001346222	8.200354896	0.000443621
Mode 2	1	1e-6	4	4	16	0.001227666	16.00389169	0.000243230
Mode 2	1	1e-8	2	4	8	0.001818777	7.700378759	0.000473449
Mode 2	1	1e-8	4	2	8	0.001200444	4.005738541	0.000717317
Mode 2	1	1e-8	4	4	16	0.001227444	4.005612311	0.000350769
Mode 2	2	1e-6	2	4	8	0.004650777	7.075686266	0.009460783
Mode 2	2	1e-6	4	2	8	0.001506333	3.233680022	0.029210002
Mode 2	2	1e-6	4	4	16	0.002162888	8.162745306	0.010171581
Mode 2	2	1e-8	2	4	8	0.008826444	4.126727756	0.015840969
Mode 2	2	1e-8	4	2	8	0.002372888	8.471389767	0.058923720
Mode 2	2	1e-8	4	4	16	0.003750000	15.02982814	0.018642591

Plots

Efficiency

Speedup

Discussion

Why MPI Dynamic performs well for irregular workloads?

MPI Dynamic performs well for irregular workloads because tasks are distributed dynamically, therefore fast workers get more work and all processes stay busy.

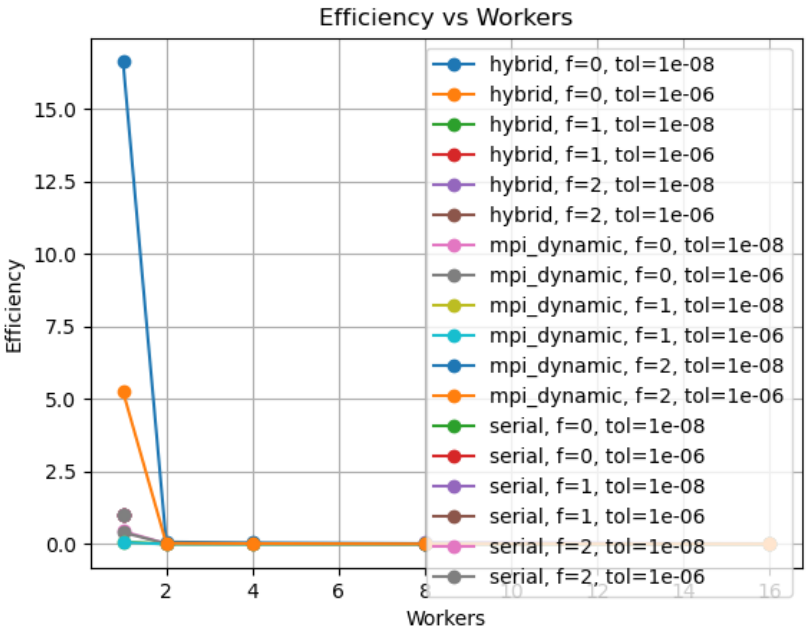


Figure 1: Mode 0 Plot

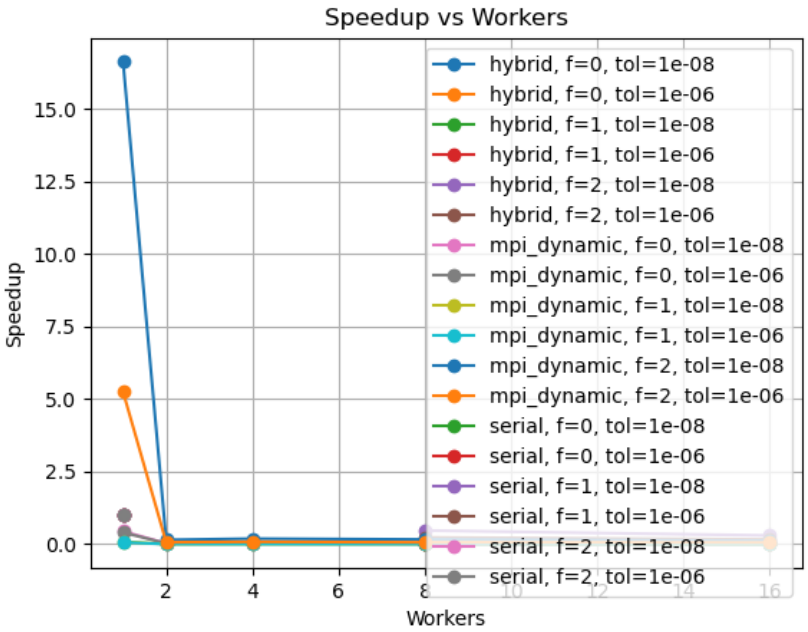


Figure 2: Mode 1 Plot

Why Hybrid might perform worse?

Hybrid works statically, ie the workers cannot send back intervals to the master to be distributed, which can lead to an imbalance in workload.

Does OpenMP compensate for the lack of MPI load balancing?

Kind of, OpenMP task stealing serves to balance threads within a process, but it cannot balance accross the MPI processes which leads to the overall imbalance still happening.

When Hybrid outperforms MPI?

Hybrid outperforms Dynamic MPI when the workload is evenly distributed as it reduces MPI communication overhead while making use of local multithreading in the MPI processes.

Impacts of thread and process count?

More threads means that local balancing gets better with less MPI communication, and parallelism leads to better parallelism but mor ecommunication overhead.